

参赛编号：004799

论文题目：B 题 城市垃圾分类运输的路径优化与调度

城市垃圾分类运输的路径优化与调度

摘要

随着我国城市化进程加速，城市生活垃圾处理面临严峻挑战。如何在满足多种垃圾分类收运要求的基础上，优化车辆调度路径、控制运输成本与碳排放，是当前亟待解决的重要问题。本文围绕三类实际问题，构建了分层约束数学模型并开发高效求解算法，使用 ArcGIS 软件将所得结果可视化。

针对问题一，在仅考虑厨余垃圾的单一运输场景下，建立了容量限制下的车辆路径优化模型（CVRP），以处理厂工作时间来限制车辆每日最大行驶时间，以最小化每日总行驶距离为目标。我们采用改进蚁群算法（IACO）并引入大规模邻域搜索策略（LNS）进行路径局部扰动优化，显著提升了解空间探索能力。通过与传统贪心算法、传统蚁群算法对比分析，在题目给定条件下，IACO-LNS 方案在保持较低时间复杂度的同时，每日运输总距离优化至 **1029.66 km**，运输车数量为 **3 辆**，表现更优。

针对问题二，模型扩展至四类车辆、四类垃圾协同运输情境，纳入各类车辆的载重、容积限制与单位运输成本约束。通过对每类垃圾分别构建子问题并沿用 IACO-LNS 算法独立求解，获得整体最优运输策略，得到最小运输成本为 **2956.89 元**。在此基础上，引入车辆“每日最大行驶时间”约束，建立带时间窗的容量限制车辆路径优化模型（CVRPTW）。模拟结果表明，时间窗约束导致部分车辆任务被拆分，运输任务重新分配，运输车数量增加，调度复杂度明显上升。

针对问题三，进一步引入中转站选址、中转站容量与时间窗、以及碳排放控制，构建“中转站选址—路径调度—碳排放最小化”的综合优化模型。采用**阶段嵌套优化模型**：外层循环枚举中转站组合，内层充分考虑中转站选址和运输的关联协同，以最小化成本为目标，使用 IACO-LNS 算法对各类垃圾的收运路径进行调度优化，并以碳排放量为扰动因子引导局部搜索。在 5 个候选站点中优选出 **3 个** 中转站组合，日均碳排放量控制在 **4009.07 kg**，十年总成本为 **1630.91 万元**，通过轮廓系数法和 PCA 综合评价验证了中转站选择与收集点分配的合理性。

为贴近城市真实交通情境，问题三进一步引入非对称路网与**禁行时间窗约束**。通过构建**方向性距离矩阵**与**动态距离函数**，模拟单行道与禁行时段影响，实现路径选择的合法性与动态绕行。更新路径代价与转移规则后，模型可实时避开不可通行路段。尽管增加了计算复杂度，但显著增强了模型对现实路网的适应能力。

综上所述，本文通过分阶段、可扩展的优化建模方法，系统考虑了实际运营中的多重约束，实现运输成本、运营效率与环境效益的协同优化，提出了一套具有实用价值的调度优化策略，为城市垃圾分类运输系统设计与优化提供理论支撑与算法参考，具备良好的应用前景与推广潜力。

关键字：城市垃圾分类运输 IACO-LNS CVRPTW 中转站选址 轮廓系数法 PCA 非对称路网 ArcGIS

一、问题重述

1.1 问题背景

随着我国城市化进程的持续推进，城市生活垃圾的产生量呈现出快速增长的趋势。根据调查统计显示，我国各城市的垃圾清运量自 2016 年至 2023 年逐年递增，2023 年已高达 4 亿吨，对城市的可持续发展和居民的健康生活构成了严峻的挑战。由于我国各城市与地区垃圾处理的能力有限，巨大的垃圾产生量使得垃圾管理问题变得越发突出，因此对我国生活垃圾的收集、运输和处理提出了更高的要求。其中垃圾分类运输是城市环境治理的一个关键环节。在治理过程中我们需要考虑不同垃圾类型的收集要求、运输车辆的载重与容积限制、中转站的处理能力及运营时间窗口。除此之外还需兼顾运输成本与碳排放控制。如何规划合理的垃圾分类运输路径与调度方案以此达到更低的成本和更高的运行效率是我们急需研究的课题。

1.2 问题重述

我们通过上述背景信息，运用数学思想，建立数学模型，解决以下问题。

问题 1：在只考虑一种垃圾类型的情况下，建立合适的数学模型，确定运输的车辆数并规划各自的运输路径和任务分配，使得车辆每日总行驶距离最小。

问题 2：考虑四种不同的垃圾类型，建立能够独立调度的多车辆协同运输模型，使得每日运输总成本最小。分析给运输车加上最长行驶时间限制后运输路径发生什么变化。

问题 3：为提升垃圾运输效率，在城区设计合适的中转站，在满足运输成本最低同时考虑较少碳排放量。考虑实际运输状况中的不对称问题并修改模型。

二、问题分析

针对问题一，在只考虑一种垃圾类型，每个垃圾站点只被经过一次的情况下，我们将运输车辆在不同垃圾站点间的运输问题简化为有容量限制的车辆路径问题（CVRP），采用蚁群算法（ACO）与贪心算法来得到每日最小行驶总距离。其中我们对蚁群算法进行改进，使用大规模邻域搜索法（LNS）对用蚁群算法得出的初始解进行优化，得到优化后的每日最小行驶距离。将得到结果进行比较，确定最优算法与结果。

针对问题二，考虑到各类垃圾的运输是相互独立的，我们将对四种不同类型垃圾的运输问题简化为四个独立的单一种类运输问题，增加参数 k 代表垃圾种类，先分别利用问题一求得的最佳算法计算单一垃圾的运输成本再求和。针对增加最大行驶时间限制

这一问题，我们修改模型，加入一最新时间约束条件，修改蚁群算法中的转移概率 $P_{ij}^k(t)$ ，得到新的运输路径，与先前路径进行比较得到时间约束对路径规划的影响。

针对问题三，我们考虑两个阶段间的协同和关联使用两层嵌套求解出最佳路径。先用外层循环枚举出所有中转站的设置情况，再用内层循环嵌套 $IACO_LNS$ 算法，将碳排放量作为扰动因素研究在碳排放量尽量少的约束条件下的最优路径及对应的中转站设置，然后进一步使用轮廓系数和 PCA 综合评分验证设计的合理性。为解决不对称问题，我们修改数学模型，建立新的距离矩阵和约束条件，并与修改前模型进行复杂度差异分析。

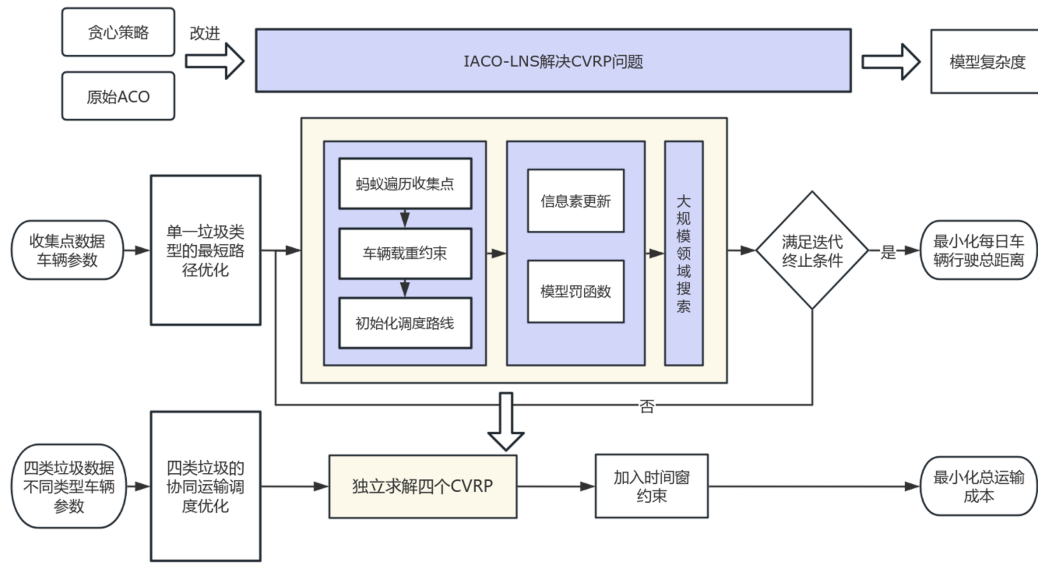


图 1 问题 1 和问题 2 流程图

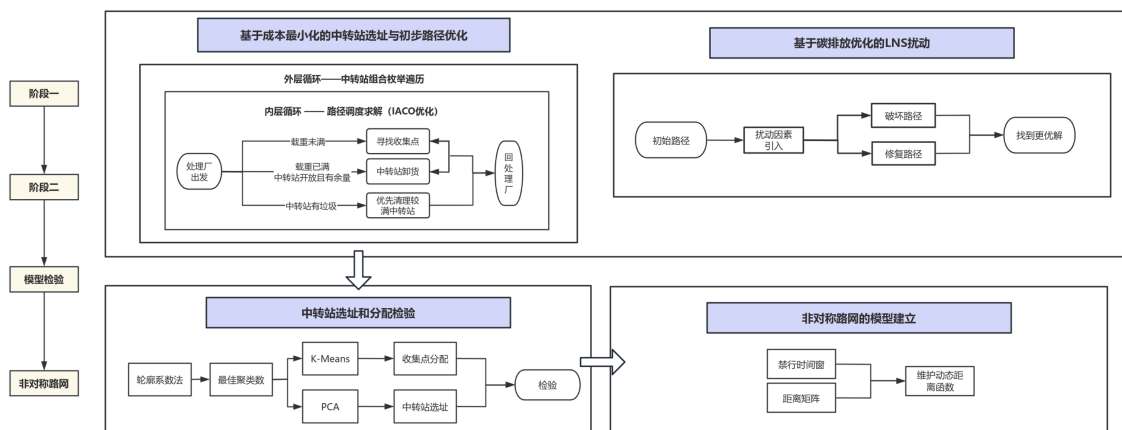


图 2 问题 3 流程图

三、模型的假设

- 假设每辆车每日行驶距离不能够超过其规定的最大的行驶距离 480km。
- 假设每辆车每日默认工作时长为 12h，从 6:00 至 18:00。
- 交通拥堵和车辆行驶速度差异不会对结果造成较大影响。
- 每日各收集点的垃圾产生规律相同，受外界影响较小。

四、符号说明

符号	含义
V	所有节点集合（含处理厂 0）
y_{ikt}	第 k 类车辆第 t 趟在点 i 的收运量
x_{ijkt}	第 k 类车辆第 t 趟是否从点 i 到 j
w_i	垃圾总产生量
Q_k	k 类运输车的最大载重
c_k	不同类型垃圾单位距离的运输成本
T_k	第 k 类车辆的出车集合
d_{ij}	ij 两点间的距离
S_k	中转站最大存储量
ol	单个样本轮廓系数
L	聚类簇
N	簇中心坐标
$[a_j, b_j]$	中转站允许车辆停靠的固定时间窗口
$[c_j, d_j]$	存在禁行时间的路段的禁行时间窗口

五、模型的建立与求解

5.1 问题一模型的建立与求解

为了解决单一垃圾种类和单一车辆的路径优化问题，我们建立如下数学模型：

- 目标函数:

$$\min \sum_{t \in T} \sum_{i \in V} \sum_{j \in V, i \neq j} d_{ij} \cdot x_{ijt}$$

- 约束条件:

- (1) 需求满足约束:

$$\sum_{t \in T} y_{it} = w_i, \quad \forall i \in N$$

- (2) 车辆容量约束:

$$\sum_{i \in N} y_{it} \leq Q, \quad \forall t \in T$$

- (3) 起终点约束:

$$\sum_{j \in N} x_{0jt} = 1, \quad \sum_{i \in N} x_{i0t} = 1, \quad \forall t \in T$$

- (4) 路径连续性约束:

$$\sum_{j \in V, j \neq i} x_{ijt} = \sum_{j \in V, j \neq i} x_{jit}, \quad \forall i \in N, \forall t \in T$$

- (5) 访问点收运量联动约束:

$$y_{it} \leq w_i \cdot \sum_{j \in V} x_{ijt}, \quad \forall i \in N, \forall t \in T$$

其中, 式(1)表示所有运输车运输垃圾量相加等于需要运输的总垃圾产生量; 式(2)表示每辆车运输垃圾量不能超过其最大载重; 式(3)表示每辆运输车的起点和终点都必须为垃圾处理厂; 式(4)表示每辆车的运输路径一定连续, 即对于每个节点 i 和每个时间段 t , 进入和离开该节点的车辆数量必须相等; 式(5)表示对于每个节点 i 和每个时间段 t , 运输量 y_{it} 不能超过该节点的需求 w_i 乘以从该节点出发的车辆数。

5.1.1.1 改进后的蚁群算法设计 (IACO-LNS)

1. 蚁群算法 (ACO)

蚁群算法是人类使用计算机来模拟自然界中蚁群觅食的行为。在选取最优觅食路径时, 前一只蚂蚁会在经过的路径留下信息素, 后一只蚂蚁则会将前一只蚂蚁所释放的信息素作为标记来选择接下来前进的路径, 以此进行多次迭代最终得到最优的觅食路径。[1] 移动规则如下:

$$j = \begin{cases} \max_{j \in N_{ik}} \{[\tau_{ij}(t)]^\alpha [\eta_{ij}(t)]^\beta\}, & r \leq r_0; \\ P_{ijk}(t), & r > r_0. \end{cases} \quad (1)$$

其中状态转移概率 $P_{ijk}(t)$ 表示在第 t 次迭代时, 蚂蚁 k 从节点 i 移动到节点 j 的概率, 可表示为

$$\frac{[\tau_{ij}(t)]^\alpha [\eta_{ij}(t)]^\beta}{\sum_{j \in N_{ik}} ([\tau_{ij}(t)]^\alpha [\eta_{ij}(t)]^\beta)};$$

N_{ik} 是未被蚂蚁访问过的节点集合, 可以通过该概率值选择下一个节点; $\tau_{ij}(t)$ 表示在第 t 次迭代时, 节点 i 与节点 j 形成的边上的信息素的量; α 表示信息素对蚂蚁搜索的影响大小; $\eta_{ij}(t)$ 表示形成的边对蚂蚁的可见度大小, $\eta_{ij}(t) = 1/d_{ij}$, d_{ij} 为节点 i 到节点 j 的距离, 如果两个节点距离越远, 表示可见度越小; β 是可见性对搜索的影响, β 越大, 蚂蚁更有可能采取可见度高的路径; r 为 $[0, 1]$ 上服从均匀分布的随机变量; r_0 为控制转移规则参数, 并且 $0 \leq r_0 \leq 1$ 。

在垃圾运输问题上, 蚁群算法具体运用如下。

令一组运输车从垃圾处理厂出发, 每一辆运输车按照式 (1) 规则选择下一访问节点 j , 当根据规则找不到合理的下一节点 j 时, 运输车返回垃圾处理厂, 该辆运输车对应的垃圾运输路径规划完成。当所有垃圾站点均被该组的某一辆运输车经过后, 得到最终的路径规划, 即可求取每日行驶总距离。

2. 大规模邻域搜索算法 (LNS) 和改进后的蚁群算法

大规模邻域搜索算法是指对已经得到的结果进行“破坏”和“修复”, 在当前解的多个邻域中寻找更满意的解, 大大提高算法在解空间的搜索范围, 避免研究对象陷入局部最优解, 从而改善初始解, 得到优质解。我们使用大规模邻域搜索算法对蚁群算法所得出的初始解进行优化。假定最优解为记录的当前路径, 然后对最优解进行“破坏”和“修复”两个阶段, 即先将随机选择一个垃圾站点, 将其从原路径中抽出, 再插入, 得到一个新的路径。计算求得新路径的目标函数值, 再其与原最优路径的目标函数值进行比较, 观察哪种方案的每日最小运输路程最小。若当前路径的总运输路程小于原最优路径的总运输路程, 则认为“破坏”“修复”后的路径更优, 更新最优解为当前路径, 否则再重复进行“破坏”“修复”两个过程, 直到输出最优路线的目标函数。

5.1.2 贪心算法

贪心算法是一种通过在每一步都选择当前可用的最优选项来解决问题的方法, 希望使用局部最优解决定全局最优解, 能够快速找到问题的可行解。但它并不考虑当前的最优结果是否会带来全局的最优结果, 即使之前有选择错误, 算法也不会逆转早期的决定。

5.1.3 时间复杂度分析

1. 蚁群算法 ($O(T \cdot A \cdot n^2)$)

蚁群算法的时间复杂度来源于在每次迭代中，有 A 只蚂蚁，每只蚂蚁构造一条完整的路径（可能包含多趟）需要遍历所有或大部分节点。选择下一个节点的计算（包括概率计算）通常涉及到对剩余 $O(n)$ 个节点的评估，因此单只蚂蚁构造完整解的复杂度近似为 $O(n^2)$ 。经过 T 轮迭代，总复杂度为 $O(T \cdot A \cdot n^2)$ 。

2. 改进后的蚁群算法 ($O(T \cdot A \cdot n^2)$)

每只蚂蚁构造初始解的复杂度仍为 $O(n^2)$ 。随后进行的 LNS 局部搜索。其“破坏”操作复杂度较低，时间复杂度主要由“修复”过程来体现。“修复”操作本质上是重新调用解的构造过程。若限定 LNS 的扰动次数为一个较小的常数，则 LNS 阶段的复杂度仍主要由多次路径重构决定，即 $O(n^2)$ 。因此，单只蚂蚁在一轮迭代中的总操作复杂度仍为 $O(n^2)$ 级别。故算法整体时间复杂度维持在 $O(T \cdot A \cdot n^2)$ 。

3. 贪心算法 ($O(n^2)$)

在每趟路径的构造中，需要从未访问的收集点集合中选择距离当前位置最近的点，这一操作的代价为 $O(n)$ 。由于每条路径最多访问 $O(n)$ 个点，且总的路径（趟次）数量最多也为 $O(n)$ 级别，因此总时间复杂度为 $O(n^2)$ 。

5.1.4 结果展示

运用上述提出的未改进的蚁群算法、改进蚁群算法和贪心算法，计算收集点为 $n=30$ 和运输车辆最大载重 $Q=5$ 吨的情况下的每日最小行驶总距离，给出具体路径，并进行比较，找到最优算法并求得最优解。

1. 结果比较

算法名称	每日最短总行驶距离 (km)	程序运行时间 (s)	时间复杂度
未改进的蚁群算法	1041.51	4.63	$O(T \cdot A \cdot n^2)$
改进后的蚁群算法	1029.66	4.35	$O(T \cdot A \cdot n^2)$
贪心算法	1062.86	0.15	$O(n^2)$

表 1 不同算法结果比较

从上述表格结果中可以看出，使用改进后的蚁群算法得到的每日最短总行驶距离最小，为 1029.66km；贪心算法的时间复杂度最低，程序运行时间只有 0.15s，但得到的结果较差。

2. 最优算法结果展示

我们使用改进后的蚁群算法得到最短总行驶距离的最优解，结果展示如下，包括运输车的数量、每辆车的具体运输路径及任务分配。

车辆编号	配送路线 (运输任务 (吨))	路径距离 (km)
1	0 → 21 (1.6) → 6 (1.5) → 25 (1.9) → 0	46.56
1	0 → 11 (1.7) → 15 (1.9) → 1 (1.2) → 2 (0.2) → 0	43.36
1	0 → 2 (2.1) → 18 (1.3) → 8 (1.1) → 24 (0.5) → 0	63.66
1	0 → 4 (3.1) → 14 (1.9) → 0	64.62
1	0 → 13 (3.2) → 20 (1.8) → 0	66.43
1	0 → 27 (1.0) → 24 (0.9) → 12 (2.1) → 28 (1.0) → 0	76.54
1	0 → 25 (0.1) → 9 (2.4) → 16 (2.5) → 0	61.72
2	0 → 19 (2.8) → 22 (2.2) → 0	75.6
2	0 → 7 (2.9) → 17 (2.1) → 0	99.27
2	0 → 29 (3.7) → 20 (1.3) → 0	72.99
2	0 → 26 (3.6) → 5 (1.4) → 0	82.73
2	0 → 14 (0.7) → 20 (0.3) → 10 (3.0) → 5 (1.0) → 0	94.96
3	0 → 5 (0.3) → 23 (3.5) → 17 (1.2) → 0	97.51
3	0 → 28 (1.3) → 3 (1.8) → 30 (1.9) → 0	83.71

表 2 改进后蚁群算法运输路径规划

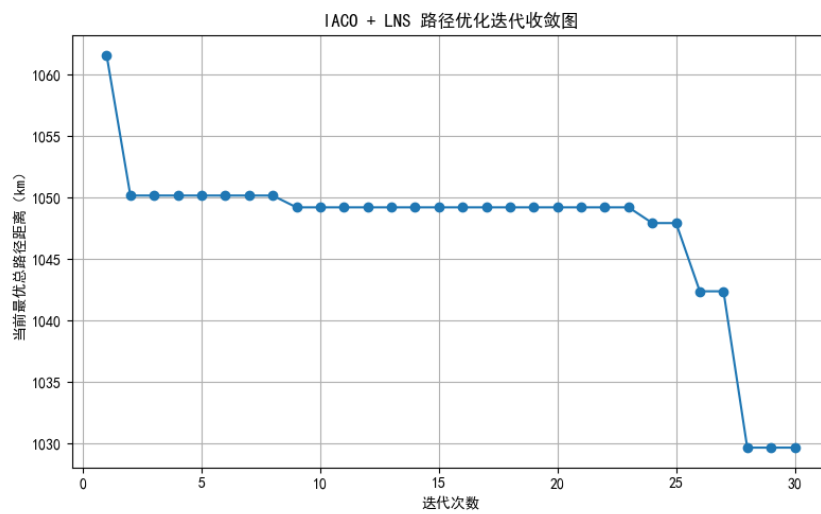


图 3 优化过程

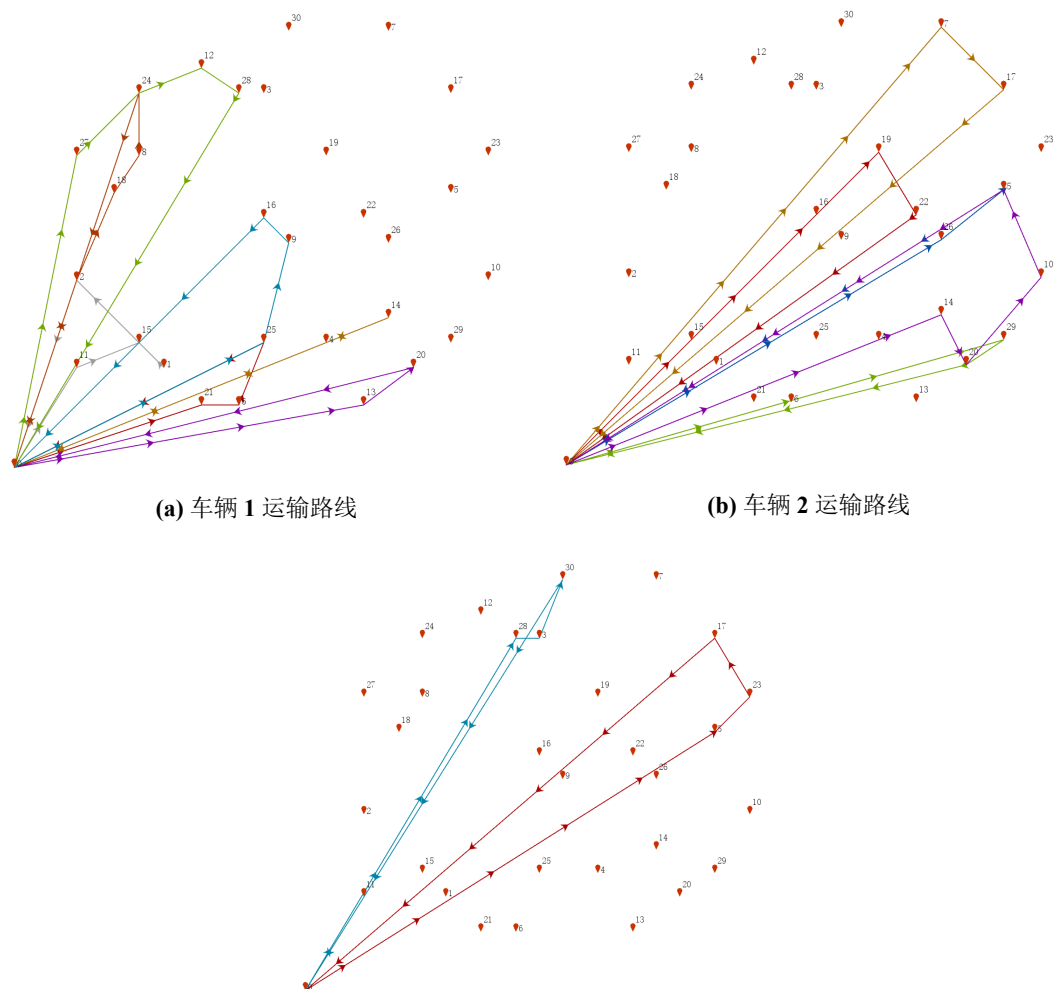


图 5 车辆 3 运输路线

可视化结果展示如上，包括每辆运输车的多趟路线以及改进后蚁群算法的路径优化迭代收敛图。

5.1.5 模型局限性

上述给出的城市垃圾分类运输的路径优化与调度模型虽然在理论上提供了有效的解决方案，但仍存在一些局限性。下面列举该模型的局限性：

- 模型中我们假设运输车的行驶速度为恒定值 40km/h ，忽略了不同车辆类型和不同路况，如交通拥堵、道路施工等随机因素影响。因此我们可以引入动态的交通信息，集成实时交通数据，应对突发状况。
- 模型中我们固定了垃圾的总产生量以及各个站点的产生量，忽略了实际问题中垃圾产生量会产生波动的情况。例如在重大节日如跨年和情人节会产生比平时更多的垃圾，需要更多的运输车参与垃圾分类和运输。
- 模型中我们将两个站点间的路径确定为两点间的直线路径，同时认为任意两点间一

定有路径连接，与该区域的实际道路图有较大的差别。我们可以根据实际地图将路径抽象出来，再使用 OD 对矩阵将其转化成实际路径网络。

5.2 问题二模型的建立与求解

- 目标函数：

$$\min \sum_{k \in K} \sum_{t \in T_k} C_k \cdot \sum_{i \in V} \sum_{j \in V, i \neq j} d_{ij} \cdot x_{ijkt}$$

- 约束条件：

(1) 各类垃圾的需求满足约束：

$$\sum_{t \in T_k} y_{ikt} = w_{ik}, \quad \forall i \in N, \forall k \in K$$

(2) 每类车辆容量约束：

$$\sum_{i \in N} y_{ikt} \leq Q_k, \quad \forall k \in K, \forall t \in T_k$$

(3) 每类车辆的起终点约束：

$$\sum_{j \in N} x_{0jkt} = 1, \quad \sum_{i \in N} x_{i0kt} = 1, \quad \forall k \in K, \forall t \in T_k$$

(4) 路径连续性约束：

$$\sum_{j \in V, j \neq i} x_{ijkt} = \sum_{j \in V, j \neq i} x_{jik t}, \quad \forall i \in N, \forall k, t$$

(5) 访问点收运量联动约束：

$$y_{ikt} \leq w_{ik} \cdot \sum_{j \in V} x_{ijkt}, \quad \forall i \in N, \forall k, t$$

我们建立与第一问类似的数学模型。与第一问不同的是，垃圾的种类增多至 4 类，于是引入参数 k ，表示第 k 类垃圾及其对应运输车辆；同时目标函数发生变化，在最小化每日总行驶距离的基础上进一步最小化每日总运输成本。

5.2.1 多车辆协同运输路线规划

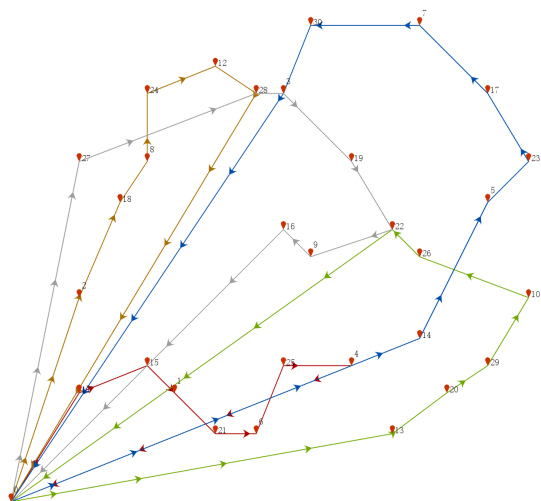
由于四类垃圾的运输相互独立，我们将四种不同类型垃圾的运输问题简化为四个独立的单一种类运输问题。使用第一问比较得到的最优算法——改进后的蚁群算法分别求出四种垃圾类型的最优运输路径，得到对应的每日最短运输距离，再乘单位距离的运输成本 C_k ，得到总运输成本。我们计算出在收集点 $n=30$ 和最大载重 $Q_k=5$ 吨情况下，四种类型垃圾的最小每日总运输成本，给出具体路径，并将结果可视化。

四类垃圾对应的总运输距离和成本如下表所示。

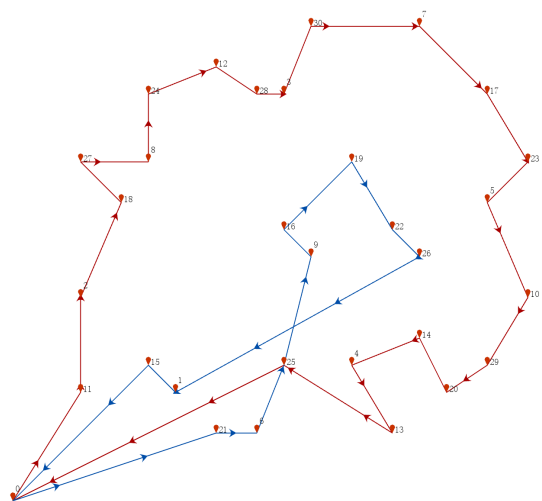
车辆类型	车辆编号	路径（运输任务（吨））	路径距离（km）	运输成本
1	1	0 → 11（1.35）→ 15（0.78）→ 1（0.72） → 21（0.91）→ 6（1.76）→ 25（1.74） → 4（0.74）→ 0	62.2	155.5
1	1	0 → 2（1.38）→ 18（1.6）→ 8（1.02） → 24（1.44）→ 12（1.87）→ 28（0.7） → 0	76.01	190.02
1	1	0 → 13（2.58）→ 20（1.08）→ 29（0.86） → 10（1.45）→ 26（1.17）→ 22（0.86） → 0	88.66	211.65
1	1	0 → 4（0.81）→ 14（1.13）→ 5（1.62） → 23（0.99）→ 17（0.72）→ 7（0.77） → 30（0.99）→ 3（0.97）→ 0	110.08	275.19
1	1	0 → 27（1.7）→ 28（1.94）→ 3（0.11） → 19（1.5）→ 22（0.04）→ 9（1.32） → 16（0.77）→ 0	91.76	229.41
2	1	0 → 11（0.27）→ 2（0.23）→ 18（0.35） → 27（0.34）→ 8（0.24）→ 24（0.24） → 12（0.51）→ 28（0.66）→ 3（0.18） → 30（0.2）→ 7（0.17）→ 17（0.27） → 23（0.27）→ 5（0.27）→ 10（0.3） → 29（0.22）→ 20（0.18）→ 14（0.21） → 4（0.31）→ 13（0.52）→ 25（0.07） → 0	144.35	288.7

车辆类型	车辆编号	路径（运输任务（吨））	路径距离（km）	运输成本
2	1	0 → 21 (0.19) → 6 (0.38) → 25 (0.25) → 9 (0.18) → 16 (0.19) → 19 (0.36) → 22 (0.2) → 26 (0.39) → 1 (0.12) → 15 (0.13) → 0	88.56	177.13
3	1	0 → 11 (0.11) → 15 (0.06) → 1 (0.06) → 21 (0.04) → 6 (0.1) → 25 (0.12) → 4 (0.06) → 14 (0.06) → 26 (0.13) → 22 (0.08) → 19 (0.09) → 16 (0.08) → 9 (0.04) → 13 (0.13) → 20 (0.09) → 29 (0.07) → 10 (0.08) → 5 (0.05) → 23 (0.07) → 17 (0.09) → 7 (0.04) → 30 (0.08) → 3 (0.04) → 28 (0.04) → 12 (0.07) → 24 (0.05) → 8 (0.07) → 18 (0.09) → 27 (0.17) → 2 (0.05) → 0	181.77	908.85
4	1	0 → 13 (1.08) → 20 (0.45) → 29 (0.65) → 10 (0.68) → 5 (0.76) → 23 (0.47) → 17 (0.72) → 7 (0.42) → 30 (0.42) → 3 (0.5) → 28 (1.06) → 12 (0.95) → 24 (0.67) → 8 (0.37) → 27 (0.8) → 0	127.51	229.52
4	1	0 → 11 (0.97) → 15 (0.32) → 1 (0.3) → 21 (0.76) → 6 (0.96) → 25 (0.72) → 4 (1.18) → 14 (0.69) → 26 (0.91) → 22 (0.33) → 19 (1.05) → 9 (0.66) → 16 (0.56) → 27 (0.39) → 18 (0.19) → 0	109.23	196.61
4	1	0 → 2 (1.38) → 18 (1.6) → 8 (1.02) → 24 (1.44) → 12 (1.87) → 28 (0.7) → 0	76.01	190.02

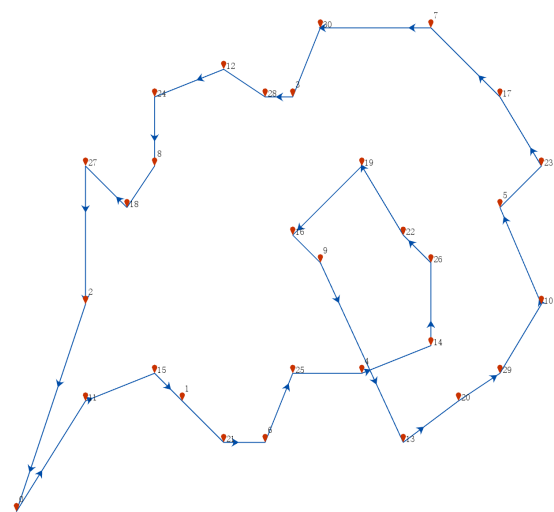
表 3 默认工作时间的多车辆运输路径



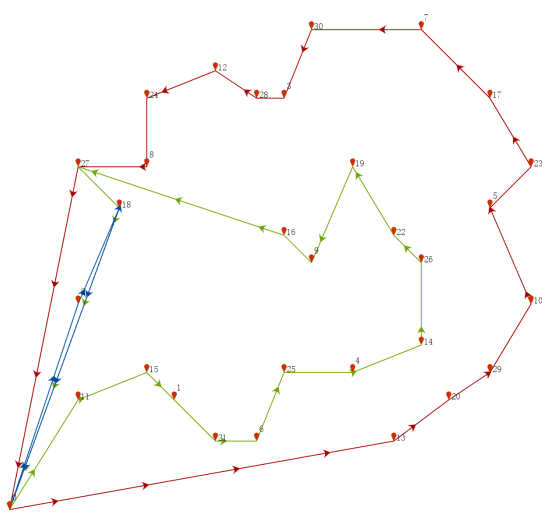
(a) 厨余垃圾运输路线



(b) 可回收物运输垃圾



(a) 有害垃圾运输路线



(b) 其他垃圾运输路线

垃圾类型	总运输距离 (km)	总运输成本 (元)
厨余垃圾	428.71	1071.77
可回收物	232.91	465.83
有害垃圾	181.77	908.85
其他垃圾	283.58	510.44
合计		2956.89

表 4 四类垃圾结果统计

5.2.2 不同时间窗口下的多车辆协调运输路线规划

将 12h 视作默认工作时长，在此基础上添加 8h 和 6h 的时间窗口，限制车辆每日最大行驶时间分别为 8h 和 6h，增加时间约束，观察其对运输车路径的影响。我们对建立的数学模型进行修改，增加一时间约束条件：

$$\sum_{t \in T_k} t_{ij} \leq T_w$$

保证运输车 k 从节点 i 转移到节点 j 的时间总和小于规定的最大工作时长。对算法的影响体现在在计算转移概率时，要增加一个二进制变量 δ_{ij}^k ，当蚂蚁 k 从节点 i 转移到节点 j 时满足时间约束时取值为 1，否则为 0。具体公式如下。

$$P_{ij}^k(t) = \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ij}(t)]^\beta \cdot \delta_{ij}^k}{\sum_{j \in N_{ik}} [\tau_{il}(t)]^\alpha \cdot [\eta_{il}(t)]^\beta \cdot \delta_{il}^k}$$

结果展示如下。

1. 最大行驶时间为 8h

车辆类型	车辆编号	路径（收集任务（吨））	路径距离	运输成本
1	1	0 → 11（1.35）→ 15（0.78）→ 1（0.72）→ 21（0.91）→ 6（1.76）→ 25（1.74）→ 4（0.74）→ 0	62.2	155.5
1	1	0 → 2（1.38）→ 18（1.6）→ 8（1.02）→ 24（1.44）→ 12（1.87）→ 28（0.7）→ 0	76.01	190.02
1	1	0 → 27（1.7）→ 28（1.94）→ 3（1.08）→ 30（0.99）→ 7（0.77）→ 17（0.72）→ 23（0.8）→ 0	87.55	218.87
1	2	0 → 13（2.58）→ 20（1.08）→ 29（0.86）→ 10（1.45）→ 5（1.62）→ 23（0.19）→ 26（0.21）→ 0	115	287.51
1	2	0 → 16（0.77）→ 9（1.32）→ 19（1.5）→ 22（0.9）→ 26（0.96）→ 14（1.13）→ 4（0.31）→ 0	86.82	217.05

表 5 8h 部分最大行驶时间路径

将上表结果与先前结果进行对比可以得到加入最长行驶时间限制后的运输路径发生了怎样的变化。可以看出，垃圾种类 1，即厨余垃圾原本由一辆运输车即可完成任务，现在则需要两辆，说明第一辆运输车因为最长行驶时间限制无法完成全部任务，任务被拆分，剩下部分由第二辆车完成。

2. 最大行驶时间为 6h

我们进一步缩短运输车的最长行驶时间，得到 6h 时的路径规划结果。由于数据量较大，我们只列出相比未加限制时发生变化的路径。结果如下。

车辆类型	车辆编号	路径（收集任务（吨））	路径距离	运输成本
...	...	...	...	...
4	1	0 → 13（1.08）→ 20（0.45）→ 29（0.65）→ 10（0.68）→ 5（0.76）→ 23（0.47）→ 17（0.72）→ 7（0.42）→ 30（0.42）→ 3（0.5）→ 28（1.06）→ 12（0.95）→ 24（0.67）→ 8（0.37）→ 27（0.8）→ 0	127.51	229.52
4	1	0 → 15（0.32）→ 1（0.3）→ 21（0.76）→ 6（0.96）→ 25（0.72）→ 4（1.18）→ 14（0.69）→ 26（0.91）→ 22（0.33）→ 9（0.66）→ 16（0.56）→ 19（1.05）→ 18（0.87）→ 27（0.39）→ 2（0.29）→ 0	112.35	202.23
4	2	0 → 11（0.97）→ 2（0.35）→ 0	32.25	58.04

表 6 6h 部分最大行驶时间路径

从上表可以看出，厨余垃圾和和其他垃圾的车辆数增多，垃圾运输任务被拆分，需要更多的运输车辆才能在最长行驶时间限制内完成垃圾运输任务。

5.3 问题三模型的建立与求解

5.3.1 中转站的选择

我们假设先从收集点收集垃圾至中转站，再从中转站运输到垃圾处理厂。[2]
我们以最小化日均总成本和碳排放为目标，建立了一个以中转站启用组合为外层决策，基于蚁群算法与局部扰动搜索的路径优化模型，对每类垃圾的收运路径进行调度。其中外层循环为中转站的组合枚举，对所有可能的中转站组合进行遍历，记录当前的中

转站集合并计算该方案下的中转站总建设成本；内层循环为使用 IACO_LNS 算法求解最优调度路径。其运行思路为，每辆运输车从处理厂出发，到达下一站点时，判断当前站点是否能够继续收集垃圾。若当前载重未满载，则寻找下一最近的可服务收集点；若运输车已接近满载，则其有两个选择：一是将垃圾运至某时间窗可行且有容量的中转站，将其当作“临时处理厂”，二是将中转站当作收集点，将车上已有垃圾运回垃圾处理厂。

根据以上分析，建立数学模型如下：

- 目标函数：

第一目标函数——运输总成本： $C = \sum_{r \in \mathcal{R}} C_k \cdot d_r + \frac{1}{3650} \sum_{j \in TS} T_j$

其中，TS 表示启用中转站集合。

第二目标函数——碳排放量估算： $E = \sum_k \sum_{\text{车辆}t} (d_{t,k} \cdot \alpha_k + \beta_k \cdot \sum_i w_{i,k,t})$

- 新增约束条件：

中转站最大存储量约束：

$$\sum_{i \in N_c} w_{i,k,t} \leq S_k, \quad \forall k, t$$

中转站固定时间窗口约束：

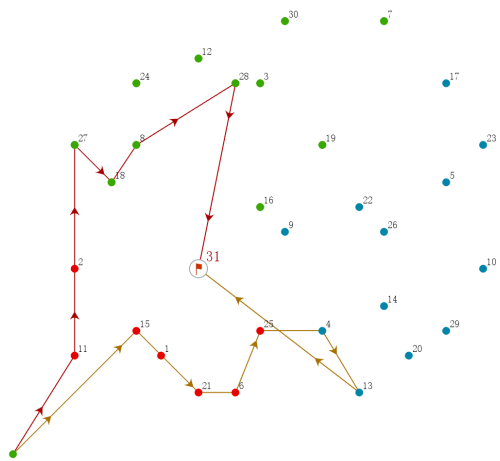
$$a_j \leq t_j \leq b_j, \quad \forall j$$

其中，中转站最大储存量约束表明每个中转站存储量不能超过其限制。若存储量超过，则无法暂存，将该站点跳过。中转站固定时间窗口约束表明车辆访问中转站的时间必须在其工作时间内，否则该中转站不运行，无法暂存垃圾。

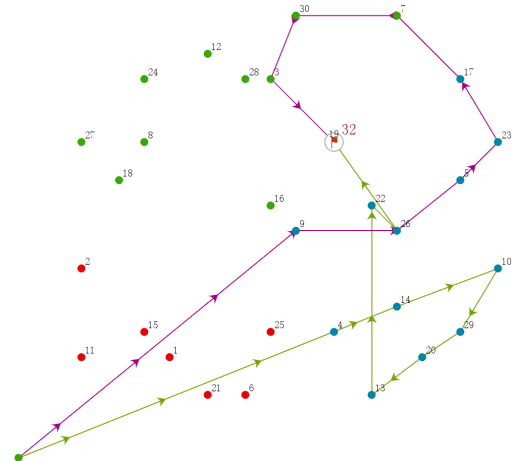
除了满足运输总成本尽量小这一目标外，我们希望碳排放量也能够尽可能小，故使用 LNS 扰动对模型进行优化，扰动因子为碳排放量，对所求的路径进行“破坏”和“修复”，在总运输成本尽可能小的前提下，得到碳排量更少的优化方案，求出最佳运输路径。我们将以上过程总结为两个阶段：第一阶段为确定中转站选址与各收集点对应的中转站分配；第二阶段为针对每个中转站，优化各类型车辆的运输路径。两阶段的关联与协同机制主要体现为阶段一所选取的方案不同，会直接影响路径中可用的转运节点；而路径成本反作用于选址评估，通过运输成本的反馈来反映该中转站组合的优劣程度。两阶段结果相互作用，形成反馈网络，多次运行后得到总成本较低且碳排放量较少的最优方案。该方案中不同种类垃圾运输至对应中转站，再从中转站运回垃圾处理厂的运输成本、收集成本、总碳排放量和总成本展示如下，并绘制出对应路线图。

日均运输成本（元）	日均收集成本（元）	日均碳排放量（千克）	十年总成本（万元）
1311.27	2704.91	4009.07	1630.9057

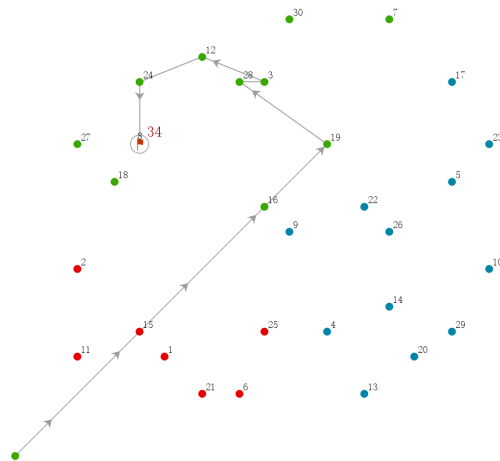
表 7 结果展示



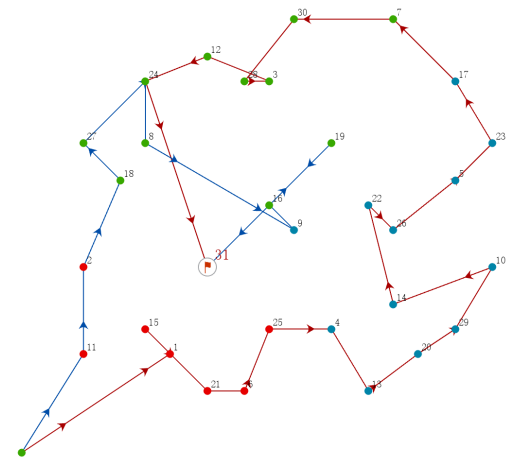
(a) 厨余垃圾到中转站 31



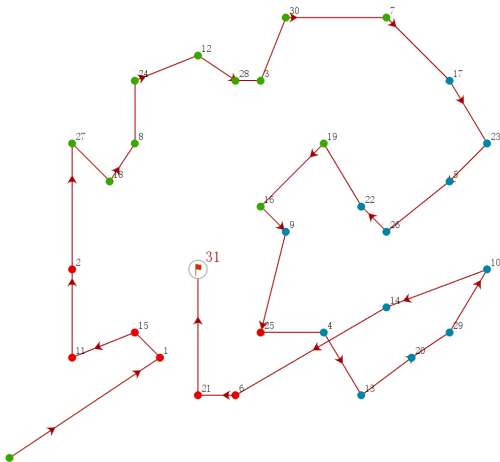
(b) 厨余垃圾到中转站 32



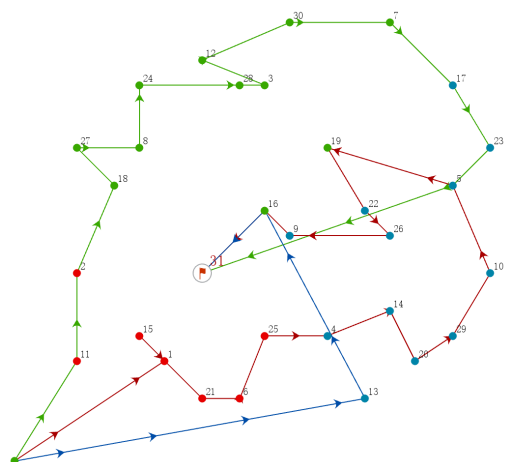
(a) 厨余垃圾到中转站 34



(b) 可回收物到中转站 32



(a) 有害垃圾到中转站 34



(b) 其他垃圾到中转站 31

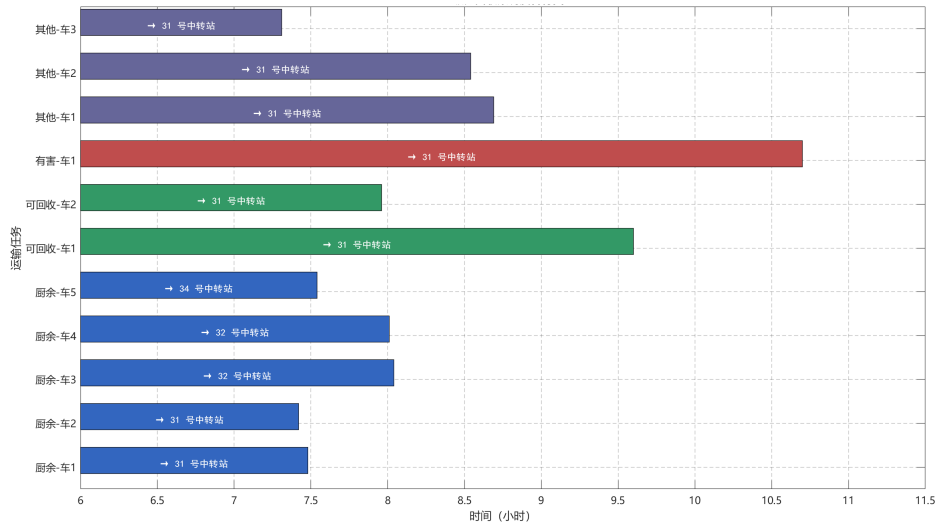


图 11 垃圾收集调度甘特图

5.3.2 中转站选择检验

1. 利用轮廓系数法检验中转站数目

轮廓分析法的目标是确定一个最优的聚类数目 k ，使得聚类结果在“簇内样本足够近，簇间样本足够远”这两个方面达到一个较好的平衡。其中轮廓系数是用来衡量数据集中的样本点与其自身簇中和其他簇中样本点的相似程度的指标，反应了聚类效果的优劣程度。轮廓系数越大说明聚类效果更好。[3]

我们根据簇内不相似度 u_i 和簇间不相似度 v_i 先计算得到单个样本点 i 的轮廓系数，再计算整体的轮廓系数。

- ol —— 单个样本点的轮廓系数

根据 $u(i)$ 和 $v(i)$ 计算样本点 i 的轮廓系数：

$$s(i) = \frac{(v(i) - u(i))}{\max(v(i), u(i))}$$

如果簇中只有一个样本，则 $ol=0$ 。 ol 的取值范围为 $[-1, 1]$ ，当取值越接近 1，说明 u_i 远小于 v_i ，即样本点 i 与其自身簇的样本点非常接近，而与其他簇的样本点距离较远，聚类效果更好；反之，当取值越接近 -1 时，样本点 i 距其他簇的样本点比自身簇的样本点更接近，样本点 i 可能被分到了错误的簇，聚类效果较差。

- 整体轮廓系数—— Ol

计算数据集中所有样本点轮廓系数的平均值，作为该聚类方案，即某特定 k 值下的整体轮廓系数：

$$Ol = \frac{1}{N} \sum_{i=1}^N ol(i)$$

在寻找最佳中转站数量这一问题中，我们能够验证当 $k=3$ 时的轮廓系数最大，即设

立 3 个中转站时各垃圾收集点的聚类效果最好。得到结果如下。

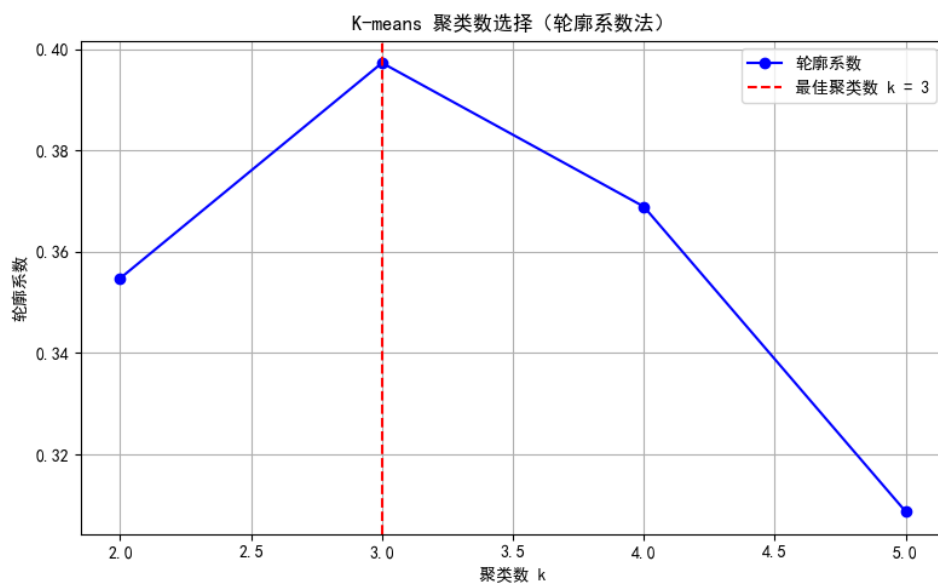


图 12 不同数目中转站的轮廓系数

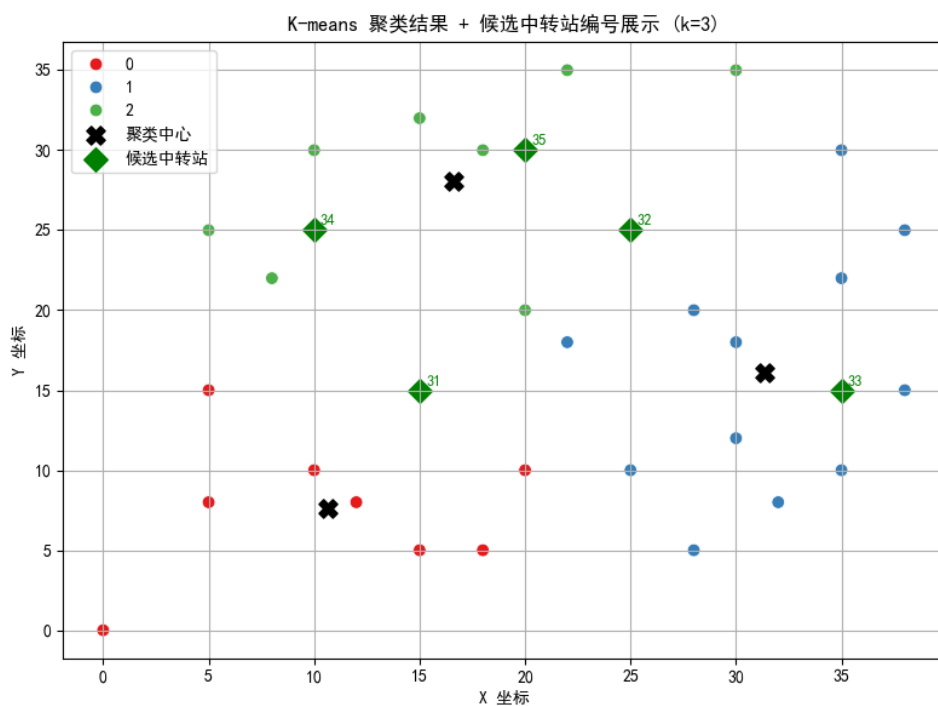


图 13 聚类结果展示

2. 使用 PCA 综合评价检验中转站选择

我们已经验证了适合的中转站数目为 3，应当所有收集点划分为三个簇。现希望分别在这三个簇中找到最合适的中转站使得垃圾运输总成本最低，故对这 5 个候选中转站进行 PCA 综合评价检验，找到每个簇中评分最高作为最终结果。我们采用聚类分析与

PCA 相结合的方法，建立了一个分阶段的多指标综合决策模型，来验证中转站选择的合理性。三个阶段的具体内容如下。

- 各中转站所属簇的确定

我们首先将这五个候选中转站分配给三个簇，分配规则如下。

设垃圾收集点集合为 $V = \{1, 2, \dots, n\}$ ，坐标为 (x_i, y_i) ，采用 K-Means 算法将其划分为 3 个簇。簇中心计算公式如下：[4]

$$N = \left(\frac{1}{|N_c|} \sum_{i \in N_c} x_i, \frac{1}{|N_c|} \sum_{i \in N_c} y_i \right)$$

其中 N_c 为簇 c 中收集点的索引集合。

对每个候选中转站 s 计算其到各簇中心的欧氏距离，并将中转站 s 分配给最近的簇：

$$d(s, c) = \sqrt{(x_s - x_c)^2 + (y_s - y_c)^2}$$

$$L = \arg \min_c d(s, c)$$

- 中转站多指标综合评价与 PCA 优选

我们对每个候选中转站根据如下四项指标进行建模与评价。

建设成本 T_s 从附件中读取各中转站的建设成本，取值越小越优。

时间窗长度 W_s $W_s = b_s - a_s$ ，其中 $[a_s, b_s]$ 为中转站允许作业时间窗，单位小时，取值越大越优。

容量利用率总和 U_s 设垃圾类型集合为 $k = \{1, 2, 3, 4\}$ ，中转站对每类垃圾的容量为 w_{sj} ，对应簇中该类垃圾的总量为 W_{cj} ，则：

$$U_s = \sum_{j \in k} \frac{W_{cj}}{C_{sj}}$$

若 $C_{sj}=0$ ，则设该项为惩罚值。容量利用率总和适中最优，过高可能会有过载风险，过低会导致资源浪费。

运输成本 C_s 简化模型中，定义为：

$$C_s = \sum_{j \in k} C_j \cdot \sum_{i \in N_c} d(i, s)$$

将上述指标矩阵 $Z \in \mathbb{R}^{m \times 4}$ 进行归一化处理：

$$Z'_{ij} = \frac{Z_{ij} - \min(Z_{.j})}{\max(Z_{.j}) - \min(Z_{.j})}$$

对标准化后的指标矩阵 Z' 应用 PCA，获得主成分得分 $PC_p(s)$ 与其方差解释比例 λ_p ，综合得分为：

$$\theta = \sum_p \lambda_p \cdot PC_p(s)$$

我们将计算得到的各中转站的 PCA 综合评分进行比较，得到评分最高的三个作为最终的中转站。

- 收集点最终分配与结果展示

中转站编号	最终得分（归一化后）
31	100.000000
32	98.559803
34	85.266362
35	79.223300
33	0.000000

表 8 各中转站的 PCA 综合评分

通过热力图可以反映出各评价指标对 PCA 综合评分的影响程度。颜色越深说明该指标对 PCA 综合评分的贡献越大。

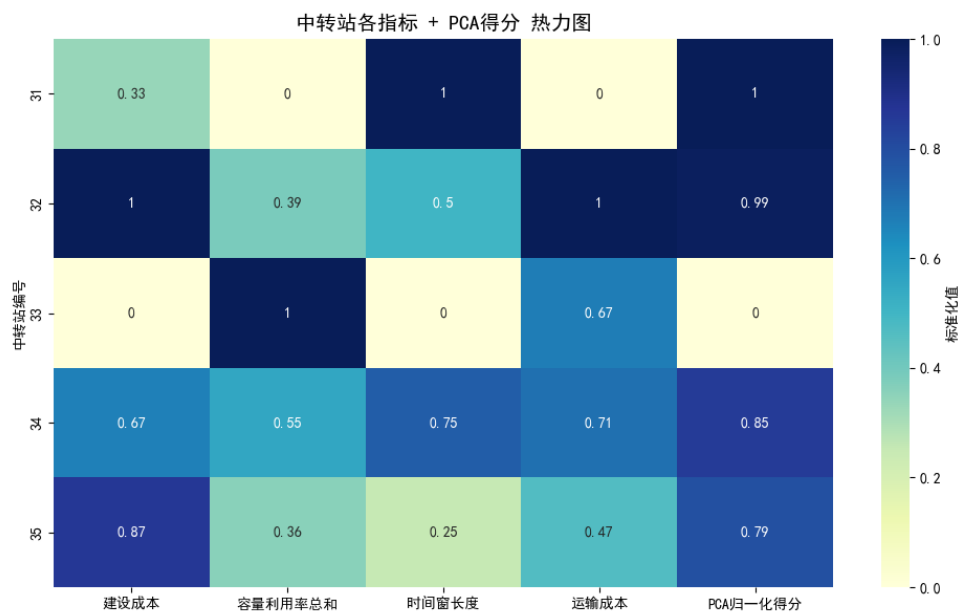


图 14 热力图

从上表的评分可以看出，31、32、34 这三个中转站的 PCA 综合评分最高，并且恰好分配在三个簇内。这一结果与我们通过蚁群算法得到的三个中转站相符合，验证了中转站选择的合理性。

5.3.3 模型调整

在实际生活中，道路会出现单行道、禁行时间段等非对称约束，即 $d_{ij} \neq d_{ji}$ ，因此我们需要构建两个距离矩阵，明确其运行方向，加以区分。

$$D = [d_{ij}]_{n \times n}, \quad D^T = [d_{ji}]$$

除此之外，我们增加一个新的约束条件——时间窗禁行约束：

$$t_{ij} \notin [c_{ij}, d_{ij}]$$

在计算 d_{ij} 时，若路径 (i, j) 属于禁行路段，则其到达时间需满足上式的约束条件，否则应使用绕行距离 d'_{ij} 。

我们将对称路网与非对称路网进行复杂度差异分析，具体分析结果如下表所示。

属性	对称路网	非对称路网	说明
路径数	$n!$	$n!$	理论路径组合数相同，但对称路径中 $i \rightarrow j$ 与 $j \rightarrow i$ 可视为一条，路径数实际减少一半。
搜索状态复杂度	$\mathcal{O}(n)$	$\mathcal{O}(n \cdot t)$	非对称路径需考虑到达时刻 t 是否满足禁行时段或方向限制。
计算复杂度（ACO）	$\mathcal{O}(n^2 \cdot k \cdot \text{iter})$	$\mathcal{O}(n^2 \cdot t \cdot k \cdot \text{iter})$	k 为蚂蚁个数， iter 为迭代次数， t 为每段路径的合法性检查时间开销。
路径可行性检查	简单	复杂	对称路网仅需检查距离和容量，非对称路网需同时判断方向性和时间窗限制。

表 9 对称与非对称路网路径优化问题的复杂度对比

六、模型评价与推广

模型的优点

- **构建了协同优化的双层结构模型：**本模型将“中转站启用组合”作为外层决策，“路径规划与任务分配”作为内层优化，通过嵌套改进蚁群算法（IACO）与局部扰动搜索（LNS）形成反馈机制，提升了解的全局性和调度的协调性。
- **综合优化碳排放与经济成本：**模型同时引入运输成本与碳排放指标，形成了“经济+环保”的双目标优化体系，符合现代城市绿色交通的发展方向；以“日均总成本最小”为主目标，兼顾碳排放最小的调度路径，有效兼容了多重约束下的综合绩效指标。
- **算法可扩展性强：**模型采用蚁群算法与 LNS 局部优化框架，便于集成更多交通要素（如突发拥堵、随机需求变化等），具备良好的可扩展性和工程适用性。
- **路径调度可解释性强：**结合路径图可视化与甘特图工具，模型输出可直观展示每日各车辆路径分配、中转站作业节奏和关键节点通勤关系，增强成果在城市调度系统中的应用潜力。
- **中转站设置合理性可验证：**结合 K-means 聚类与 PCA 综合评价方法，对最终选定的中转站布局进行“聚类-选址”双重校验，提升了站点设置的科学性与数据解释能力。

模型的局限性

- **未考虑垃圾产生的动态波动：**模型默认每日垃圾产生量恒定，未引入季节、节假日等时间因素带来的需求波动，后续可引入时间序列预测模型增强模型动态适应能力。
- **对交通状况简化处理：**当前版本中车辆速度设为常数，忽略了交通高峰、道路封闭等突发因素，未来可引入动态 OD 矩阵或高德路况数据进行实时更新与调度。
- **碳排放模型为线性估计：**碳排放估算采用了线性距离函数，未引入发动机非线性特性和停靠损耗等更真实的排放机制。
- **路径生成仍为启发式近似解：**虽然结合了蚁群+LNS 增强搜索深度，但对较大规模站点仍存在解空间收敛慢或精度波动的风险，后续可尝试遗传算法或强化学习路径优化机制。

模型的推广应用

- **适用于城市固废物流与应急调度：**本模型可直接扩展用于城市大件垃圾回收、建筑废弃物清运等任务，具有普适性的空间调度框架。
- **可用于城市智能交通系统中：**通过嵌入智慧城市的交通大数据平台，可与城市导航调度系统对接，实现垃圾运输系统与公共交通、物流配送的协同管理。

- **支持非对称与高时效路径问题：**模型调整后可容纳单行道、禁行时间窗等现实因素，适用于城市配送、电动物流车调度等领域的复杂路径规划任务。
- **可扩展为碳配额政策模拟工具：**通过引入排放因子动态调节机制，可用于城市低碳试点区域，测试不同中转布局对城市碳排放总量控制的政策模拟作用。
- **具备推广至其他城市的通用能力：**模型输入为标准格式坐标与属性表格，易于迁移部署至其他城市中，作为基础垃圾调度优化模块在“智慧环卫系统”中广泛应用。

参考文献

- [1] 唐传茵, 章明理, 李静红, 苑莹, 卫美荣. 基于改进蚁群算法的外卖配送路径规划研究 [J]. 南京信息工程大学学报 (自然科学版), 2024, 16(2): 145–154. DOI: 10.13878/j.cnki.jnuist.20230311001.
- [2] 李婷婷. 城市生活垃圾收运车辆动态调度与路径优化方法研究 [D]. 扬州大学, 2024. DOI:10.27441/d.cnki.gyzdu.2024.000831.
- [3] 刘明辉. L 县农村垃圾分类回收路径优化及中转站选址研究 [D]. 北京: 北京交通大学, 2021. DOI: 10.26944/d.cnki.gbfju.2021.003368.
- [4] 周红丽. M 企业 C 仓快递中转站选址及配送路径规划改善研究 [D]. 重庆大学, 2022. DOI: 10.27670/d.cnki.gcqdu.2022.001938.

附录 A 支撑材料

以下是本文模型搭建过程中所用的主要数据与代码支撑材料列表：

表 10 支撑材料列表

文件名	作用描述
GIS 数据集	GIS 数据集文件夹
Q1/OD 对生成.py	距离矩阵 OD 对的生成代码
Q1/GREEDY.py	采用贪心算法的基础路径优化代码
Q1/ACO.py	采用普通蚁群算法的基础路径优化代码
Q1/IACO-LNS.py	采用改进蚁群算法的基础路径优化代码
Q1/收敛过程图.png	IACO 的收敛过程图
Q1/shp.py	IACO 的最佳路径 shp 文件生成代码
Q2/四类 IACO-TW.py	加上时间窗口的四类 IACO 的路径优化代码
Q2/四类 shp.py	四类 IACO 最佳路径 shp 文件生成代码
Q2/四类 IACO-TW-12.xlsx	12 个小时限制下的最优路线表
Q2/四类 IACO-TW-8.xlsx	8 个小时限制下的最优路线表
Q2/四类 IACO-TW-6.xlsx	6 个小时限制下的最优路线表
Q3/收集.py	综合 IACO 的中转站选址和收集路径规划代码
Q3/运输.py	综合 IACO 的运输路径规划代码
Q3/最优收集路线.xlsx	最优收集路线细则表
Q3/最优运输路线.xlsx	最优运输路线细则表
Q3/收集点对应中转站.xlsx	各收集点对应的中转站分配表
Q3/收集 shp	最优收集路径的 shp 文件生成代码
Q3/运输 shp	最优运输路径的 shp 文件生成代码
Q3/轮廓系数法.py	轮廓系数法验证代码
Q3/聚类 +PCA.py	基于轮廓系数的 K-means 聚类及 PCA 评估代码
Q3/PCA.m	PCA 相关参数图绘制代码
Q3/gante.m	最优收集方案甘特图绘制代码

附录 B 关键 Python 代码

```
1 import pandas as pd
2 import numpy as np
3 from math import sqrt
4 import time
5 start_time = time.time()
6
7 # ==== 参数设置 ====
8 capacity = 5 # 吨
9 speed = 40 # km/h
10 work_time = 12 # 小时
11 max_distance = speed * work_time
12
13 # ==== 数据读取 ====
14 df = pd.read_excel("收集点坐标.xlsx")
15 points = df[['X', 'Y']].values
16 weights = df['W'].values.copy()
17 n = len(points)
18
19 # ==== 距离矩阵 ====
20 def euclidean_distance(a, b):
21     return sqrt((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2)
22
23 distance_matrix = np.array([[euclidean_distance(points[i], points[j]) for j in range(n)] for i
24                               in range(n)])
25
26 # ==== 贪心构造解 ====
27 def construct_greedy_solution():
28     remaining = weights.copy()
29     vehicle_id = 0
30     plan = []
31
32     while remaining.sum() > 1e-6:
33         vehicle_id += 1
34         trip_id = 0
35         vehicle_distance = 0
36
37         while remaining.sum() > 1e-6 and vehicle_distance < max_distance:
38             route = [0]
39             load = 0
40             trip_distance = 0
41             delivered = np.zeros(n)
42             visited = set()
43
44             while True:
```

```

44     current = route[-1]
45     candidates = [i for i in range(1, n) if remaining[i] > 0 and i not in visited]
46     if not candidates:
47         break
48     # 贪心选择最近未访问点
49     next_point = min(candidates, key=lambda j: distance_matrix[current][j])
50     travel = distance_matrix[current][next_point]
51     return_home = distance_matrix[next_point][0]
52
53     if trip_distance + travel + return_home + vehicle_distance > max_distance:
54         break
55
56     deliverable = min(capacity - load, remaining[next_point])
57     if deliverable <= 1e-6:
58         visited.add(next_point)
59         continue
60
61     route.append(next_point)
62     load += deliverable
63     trip_distance += travel
64     delivered[next_point] += deliverable
65     remaining[next_point] -= deliverable
66     visited.add(next_point)
67
68     if load >= capacity:
69         break
70
71     if len(route) == 1:
72         break
73
74     route.append(0)
75     trip_distance += distance_matrix[route[-2]][0]
76     vehicle_distance += trip_distance
77
78     plan.append({
79         "车辆编号": vehicle_id,
80         "趟次编号": trip_id + 1,
81         "路径": " → ".join(map(str, route)),
82         "运输任务": delivered,
83         "路径距离 (km)": round(trip_distance, 2)
84     })
85     trip_id += 1
86     return plan
87
88 # ==== 执行贪心算法 ====
89 plan = construct_greedy_solution()
90

```

```

91 # ==== 输出结果 ====
92 rows = []
93 total_distance = 0
94 for entry in plan:
95     delivered_list = entry["运输任务"]
96     delivered_str = ", ".join(f"{i}:{round(delivered_list[i], 2)}" for i in
97                                range(len(delivered_list)) if delivered_list[i] > 0)
98     rows.append({
99         "车辆编号": entry["车辆编号"],
100         "趟次编号": entry["趟次编号"],
101         "路径": entry["路径"],
102         "运输任务（收集点:吨）": delivered_str,
103         "路径距离（km）": entry["路径距离（km）"]
104     })
105     total_distance += entry["路径距离（km）"]
106
107 result_df = pd.DataFrame(rows)
108 result_df.to_excel("Greedy.xlsx", index=False)
109
110 print("贪心路径调度结果已保存为 'Greedy.xlsx'")
111 print(f"贪心算法总路径距离为: {total_distance:.2f} km")
112
113 end_time = time.time()
114 print(f"程序运行时间: {end_time - start_time:.2f} 秒")

```

Listing 1: 贪心算法源代码

```

1 import pandas as pd
2 import numpy as np
3 import random
4 from math import sqrt, inf
5 import time
6 import matplotlib.pyplot as plt
7 plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体
8 plt.rcParams['axes.unicode_minus'] = False # 正常显示负号
9
10
11 start_time = time.time()
12
13 # ==== 参数设置 ====
14 capacity = 5 # 每辆车最大载重（吨）
15 speed = 40 # 车速 km/h
16 work_time = 12 # 工作时间 h
17 max_distance = speed * work_time # 每辆车单日最大行驶距离
18
19 num_ants = 10
20 num_iterations = 30

```

```

21 alpha = 1.0
22 beta = 5.0
23 rho = 0.1
24 Q = 100.0
25
26 # ==== 数据读取 ====
27 df = pd.read_excel("收集点坐标.xlsx")
28 points = df[['X', 'Y']].values
29 demands = df['W'].values
30 n = len(points)
31
32 # ==== 距离与信息素矩阵 ====
33 distance_matrix = np.array([[sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2) for x2, y2 in points] for
    x1, y1 in points])
34 eta = 1 / (distance_matrix + 1e-6)
35 pheromone = np.ones((n, n))
36
37 # ==== 构造蚁群解 ====
38 def construct_solution():
39     remaining = demands.copy()
40     vehicle_id = 0
41     plan = []
42
43     while remaining.sum() > 1e-6:
44         vehicle_id += 1
45         vehicle_distance = 0
46         trip_id = 0
47         while vehicle_distance < max_distance and remaining.sum() > 1e-6:
48             route = [0]
49             load = 0
50             trip_distance = 0
51             delivered = np.zeros(n)
52             visited = []
53
54             while True:
55                 current = route[-1]
56                 candidates = [i for i in range(1, n) if remaining[i] > 0 and i not in visited]
57                 if not candidates:
58                     break
59                 probs = [(pheromone[current][j] ** alpha) * (eta[current][j] ** beta) for j in
                    candidates]
60                 total = sum(probs)
61                 if total == 0:
62                     break
63                 probs = [p / total for p in probs]
64                 next_point = random.choices(candidates, probs)[0]
65                 travel = distance_matrix[current][next_point]

```

```

66         return_home = distance_matrix[next_point][0]
67         if trip_distance + travel + return_home + vehicle_distance > max_distance:
68             break
69         deliverable = min(capacity - load, remaining[next_point])
70         if deliverable <= 1e-6:
71             visited.append(next_point)
72             continue
73         route.append(next_point)
74         load += deliverable
75         trip_distance += travel
76         delivered[next_point] += deliverable
77         remaining[next_point] -= deliverable
78         visited.append(next_point)
79
80     if len(route) == 1:
81         break
82
83     route.append(0)
84     trip_distance += distance_matrix[route[-2]][0]
85     vehicle_distance += trip_distance
86     plan.append({
87         "车辆编号": vehicle_id,
88         "趟次编号": trip_id + 1,
89         "路径": " → ".join(map(str, route)),
90         "运输任务": delivered,
91         "路径距离": round(trip_distance, 2)
92     })
93     trip_id += 1
94     return plan
95
96 # ==== LNS 局部搜索 ====
97 def lns_improve(plan):
98     best_plan = plan.copy()
99     best_cost = sum(p["路径距离"] for p in best_plan)
100     for _ in range(5):
101         temp_plan = best_plan.copy()
102         if len(temp_plan) < 2:
103             break
104         idx = random.randint(0, len(temp_plan) - 1)
105         temp_plan.pop(idx)
106         new_subplan = construct_solution()
107         merged_plan = temp_plan + new_subplan
108         merged_cost = sum(p["路径距离"] for p in merged_plan)
109         if merged_cost < best_cost:
110             best_cost = merged_cost
111             best_plan = merged_plan
112     return best_plan

```

```

113
114 # ==== IACO + LNS 主循环 ====
115 best_overall_plan = None
116 best_total_distance = inf
117 best_distances = []
118
119 for iteration in range(num_iterations):
120     all_solutions = [construct_solution() for _ in range(num_ants)]
121     for plan in all_solutions:
122         plan = lns_improve(plan)
123         total_dist = sum(p["路径距离"] for p in plan)
124         if total_dist < best_total_distance:
125             best_total_distance = total_dist
126             best_overall_plan = plan
127     best_distances.append(best_total_distance)
128     pheromone *= (1 - rho)
129     for plan in all_solutions:
130         for p in plan:
131             nodes = [int(i) for i in p["路径"].split(" → ")]
132             for i in range(len(nodes) - 1):
133                 pheromone[nodes[i]][nodes[i + 1]] += Q / (p["路径距离"] + 1e-6)
134
135 # ==== 输出 Excel ====
136 output = []
137 for p in best_overall_plan:
138     delivery = p["运输任务"]
139     delivery_str = ", ".join(f"{i}:{round(delivery[i], 2)}" for i in range(len(delivery)) if
140                               delivery[i] > 0)
141     output.append({
142         "车辆编号": p["车辆编号"],
143         "趟次编号": p["趟次编号"],
144         "路径": p["路径"],
145         "运输任务（收集点:吨）": delivery_str,
146         "路径距离（km）": p["路径距离"]
147     })
148
149 output_df = pd.DataFrame(output)
150 output_df.to_excel("IACO_LNS.xlsx", index=False)
151 print("已生成最优路径表格: IACO_LNS.xlsx")
152 print(f"IACO + LNS 总路径距离为: {best_total_distance:.2f} km")
153
154 # ==== 绘制收敛过程图 ====
155 plt.figure(figsize=(8, 5))
156 plt.plot(range(1, num_iterations + 1), best_distances, marker='o')
157 plt.xlabel("迭代次数")
158 plt.ylabel("当前最优总路径距离（km）")
159 plt.title("IACO + LNS 路径优化迭代收敛图")

```



```

159 plt.grid(True)
160 plt.tight_layout()
161 plt.savefig("收敛过程图.png")
162 plt.show()
163
164 end_time = time.time()
165 print(f"程序运行时间: {end_time - start_time:.2f} 秒")

```

Listing 2: 改进蚁群算法源代码

```

1  import pandas as pd
2  import numpy as np
3  import itertools
4  import random
5  from math import sqrt, inf
6  from copy import deepcopy
7
8  # ==== 参数 ====
9  speed = 40
10 max_time = 12
11 max_distance = speed * max_time
12 alpha = 1.0
13 beta = 5.0
14 rho = 0.1
15 Q = 100.0
16 num_ants = 10
17 num_iterations = 15
18 max_enabled_stations = 3
19 start_hour = 6.0
20
21 # ==== 数据读取 ====
22 points_df = pd.read_excel("收集点信息.xlsx")
23 stations_df = pd.read_excel("中转站候选列表.xlsx")
24 vehicles_df = pd.read_excel("4类运输车辆参数.xlsx")
25 emission_df = vehicles_df
26
27 emission_alpha = emission_df["碳排放系数"].values
28 emission_beta = emission_df["碳排放系数"].values
29
30 # ==== 预处理 ====
31 collect_coords = points_df[['X', 'Y']].values
32 station_coords = stations_df[['X', 'Y']].values
33 facility_coord = np.array([[0, 0]])
34 all_coords = np.vstack([facility_coord, collect_coords, station_coords])
35 distance_matrix = np.array([
36     [sqrt((x1 - x2)**2 + (y1 - y2)**2) for (x2, y2) in all_coords]
37     for (x1, y1) in all_coords]

```

```

38 ])
39
40 w_all = points_df[['厨余垃圾wi1', '可回收物 wi2', '有害垃圾 wi3', '其他垃圾wi4']].values
41 Qk = vehicles_df['载重Qk'].values
42 Ck = vehicles_df['距离成本Ck'].values
43 station_T = stations_df['T'].values
44 station_TW = stations_df['TW'].apply(lambda x: eval(str(x)) if isinstance(x, str) else
    x).values
45 station_S = stations_df['S'].apply(lambda x: eval(str(x)) if isinstance(x, str) else x).values
46
47 n_collect = len(collect_coords)
48 n_station = len(station_coords)
49 station_ids = list(range(n_station))
50
51 # ==== 中转站组合 ====
52 station_combinations = []
53 for r in range(1, max_enabled_stations + 1):
54     station_combinations += list(itertools.combinations(station_ids, r))
55
56 best_total_cost = inf
57 best_plan_all = []
58 best_total_emission = inf
59
60 # ==== 主循环 ====
61 for enabled in station_combinations:
62     build_cost = sum(station_T[j] for j in enabled) / 3650
63     total_transport_cost = 0
64     total_emission = 0
65     plan_all = []
66
67     for t in range(4):
68         demands = w_all[:, t].copy()
69         capacity = Qk[t]
70         station_cap = {j: station_S[j][t] for j in enabled}
71         pheromone = np.ones((1 + n_collect + n_station, 1 + n_collect + n_station))
72         eta = 1 / (distance_matrix + 1e-6)
73
74         def construct_solution():
75             remaining = demands.copy()
76             station_used = deepcopy(station_cap)
77             plan = []
78             vehicle_id = 0
79
80             while remaining.sum() > 1e-6:
81                 vehicle_id += 1
82                 vehicle_distance = 0
83                 trip_id = 0

```

```

84
85     while vehicle_distance < max_distance and remaining.sum() > 1e-6:
86         route = [0]
87         load = 0
88         trip_distance = 0
89         visited = []
90         delivered = np.zeros(n_collect)
91         mid_station_record = None
92
93     while True:
94         current = route[-1]
95         candidates = [i for i in range(1, 1 + n_collect)
96                      if remaining[i - 1] > 0 and i not in visited]
97         if not candidates:
98             break
99         probs = [(pheromone[current][j] ** alpha) * (eta[current][j] ** beta)
100                for j in candidates]
101         total = sum(probs)
102         if total == 0:
103             break
104         probs = [p / total for p in probs]
105         next_point = random.choices(candidates, probs)[0]
106         travel = distance_matrix[current][next_point]
107         return_home = distance_matrix[next_point][0]
108         if trip_distance + travel + return_home + vehicle_distance > max_distance:
109             break
110         deliverable = min(capacity - load, remaining[next_point - 1])
111         if deliverable <= 1e-6:
112             visited.append(next_point)
113             continue
114         route.append(next_point)
115         load += deliverable
116         trip_distance += travel
117         delivered[next_point - 1] += deliverable
118         remaining[next_point - 1] -= deliverable
119         visited.append(next_point)
120
121     if len(route) == 1:
122         break
123
124     sent_to_station = False
125     for sj in enabled:
126         sj_idx = 1 + n_collect + sj
127         real_id = int(stations_df.iloc[sj]["i"])
128         sj_time = station_TW[sj]
129         sj_cap = station_used[sj]
130         arrival_time = start_hour + (trip_distance +

```

```

distance_matrix[route[-1]][sj_idx]) / speed

131
132     if sj_cap >= load and sj_time[0] <= arrival_time <= sj_time[1]:
133         route.append(sj_idx)
134         trip_distance += distance_matrix[route[-2]][sj_idx]
135         vehicle_distance += trip_distance
136         station_used[sj] -= load
137         mid_station_record = {
138             "id": real_id,
139             "arrival_time": round(arrival_time, 2),
140             "unload": round(load, 2),
141             "remaining": round(station_used[sj], 2)
142         }
143         load = 0
144         sent_to_station = True
145         break
146
147     if not sent_to_station:
148         route.append(0)
149         trip_distance += distance_matrix[route[-2]][0]
150         vehicle_distance += trip_distance
151
152     translated_route = []
153     for node in route:
154         if node == 0:
155             translated_route.append(0)
156         elif 1 <= node <= n_collect:
157             real_id = int(points_df.iloc[node - 1]['i'])
158             translated_route.append(real_id)
159         else:
160             station_index = node - 1 - n_collect
161             if 0 <= station_index < len(stations_df):
162                 real_id = int(stations_df.iloc[station_index]['i'])
163                 translated_route.append(real_id)
164             else:
165                 translated_route.append(node)
166
167     d = trip_distance
168     = emission_alpha[t]
169     = emission_beta[t]
170     total_weight = np.sum(delivered)
171     emission = d * + d * * total_weight
172
173     plan.append({
174         "垃圾类型": t + 1,
175         "车辆编号": vehicle_id,
176         "趟次编号": trip_id + 1,

```

```

177         "路径": " → ".join(map(str, translated_route)),
178         "路径距离 (km)": round(d, 2),
179         "运输成本": round(d * Ck[t], 2),
180         "碳排放量 (kg)": round(emission, 2),
181         "中转站": mid_station_record["id"] if mid_station_record else "无",
182         "到达时间 (h)": mid_station_record["arrival_time"] if mid_station_record
183         else "-",
184         "卸货量 (吨)": mid_station_record["unload"] if mid_station_record else
185         "-",
186         "卸后剩余容量": mid_station_record["remaining"] if mid_station_record else
187         "-"
188     })
189     trip_id += 1
190     return plan
191
192 def lns_improve(plan, vehicle_type_index):
193     best_plan = deepcopy(plan)
194     def calc_emission(p):
195         d = p["路径距离 (km)"]
196         = emission_alpha[vehicle_type_index]
197         = emission_beta[vehicle_type_index]
198         return d * + d * * 1 # 粗估平均1吨
199     best_emission = sum(calc_emission(p) for p in best_plan)
200     for _ in range(5):
201         temp_plan = deepcopy(best_plan)
202         if len(temp_plan) < 2:
203             break
204         temp_plan.pop(random.randint(0, len(temp_plan) - 1))
205         temp_plan += construct_solution()
206         temp_emission = sum(calc_emission(p) for p in temp_plan)
207         if temp_emission < best_emission:
208             best_emission = temp_emission
209             best_plan = temp_plan
210     return best_plan
211
212 best_cost = inf
213 best_plan = None
214 for _ in range(num_iterations):
215     all_plans = [lns_improve(construct_solution(), t) for _ in range(num_ants)]
216     for plan in all_plans:
217         cost = sum(p["运输成本"] for p in plan)
218         if cost < best_cost:
219             best_cost = cost
220             best_plan = plan
221     pheromone *= (1 - rho)
222     for plan in all_plans:
223         for p in plan:

```

```

221         nodes = [int(i) for i in p["路径"].split(" → ")]
222         for i in range(len(nodes) - 1):
223             pheromone[nodes[i]][nodes[i + 1]] += Q / (p["路径距离 (km)"] + 1e-6)
224
225         total_transport_cost += best_cost
226         total_emission += sum(p["碳排放量 (kg)"] for p in best_plan)
227         plan_all += best_plan
228
229         total_cost = total_transport_cost + build_cost
230         if total_cost < best_total_cost:
231             best_total_cost = total_cost
232             best_plan_all = plan_all
233             best_total_emission = total_emission
234             best_enabled_station_ids = [int(stations_df.iloc[j]["i"]) for j in enabled]
235
236         # ==== 输出 ====
237         for row in best_plan_all:
238             row["启用中转站组合"] = str(best_enabled_station_ids)
239
240         df_result = pd.DataFrame(best_plan_all)
241         df_result.to_excel("最优运输方案_碳排放优化.xlsx", index=False)
242         print("最优方案已保存 (碳排放优化)")
243         print("启用中转站组合:", best_enabled_station_ids)
244         print("最小日均总成本:", round(best_total_cost, 2), "元")
245         print("总碳排放量:", round(best_total_emission, 2), "kg")

```

Listing 3: 综合 IACO 的中转站选址和收集路径规划源代码

```

1  from sklearn.metrics import silhouette_score
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from sklearn.cluster import KMeans
6  import matplotlib
7  matplotlib.rcParams['font.sans-serif'] = ['SimHei']
8  matplotlib.rcParams['axes.unicode_minus'] = False
9
10 df = pd.read_excel("附件1 - 副本.xlsx")
11 print("列名: ", df.columns.tolist())
12
13 # 提取编号和坐标
14 collection_ids = df.iloc[:, 0].values # 编号
15 coords = df.iloc[:, 1:3].dropna().values # X, Y 坐标
16
17 # 计算轮廓系数以代替 SSE
18 silhouette_scores = []
19 k_range = range(2, 6)

```

```

20
21 for k in k_range:
22     kmeans = KMeans(n_clusters=k, random_state=42).fit(coords)
23     score = silhouette_score(coords, kmeans.labels_)
24     silhouette_scores.append(score)
25
26 # 找到轮廓系数最大的 k
27 best_k_silhouette = k_range[np.argmax(silhouette_scores)]
28
29 # 绘图
30 plt.figure(figsize=(8, 5))
31 plt.plot(k_range, silhouette_scores, 'o-', color='blue', label="轮廓系数")
32 plt.axvline(x=best_k_silhouette, color='red', linestyle='--', label=f"最佳聚类数 k =
    {best_k_silhouette}")
33 plt.xlabel('聚类数 k')
34 plt.ylabel('轮廓系数')
35 plt.title('K-means 聚类数选择（轮廓系数法）')
36 plt.legend()
37 plt.grid(True)
38 plt.tight_layout()
39 plt.show()

```

Listing 4: 轮廓系数法验证源代码

附录 C 人工智能辅助工具使用说明

工具名称：OpenAI ChatGPT（2023 年 11 月 5 日版本，GPT-4 模型）

使用时间：2025 年 5 月 24 日

使用目的：辅助模型编程与原理解释

提问 1：请介绍蚁群算法的基本原理。

输出内容：

蚁群算法（Ant Colony Optimization, ACO）是一种模拟自然界中蚂蚁寻找食物路径行为的优化算法。蚂蚁在路径中释放信息素，后续蚂蚁根据信息素浓度进行路径选择，从而在多次迭代中找到最短或最优路径。该算法通过状态转移概率控制路径探索，并结合信息素挥发机制避免局部最优，适用于组合优化、路径规划等问题。

提问 2：如何根据最优路线表生成对应的 shp 文件？

输出内容：

使用 pandas 和 geopandas 读取最优路径 Excel 表格，解析“路径”列中的节点序列，结合坐标表建立 LineString 几何对象，构建 GeoDataFrame 后使用 to_file() 方法批量

导出为 shapefile 文件，实现路径数据在 GIS 平台中的空间可视化。

提问 3：算法复杂度的定义是什么？

输出内容：

算法复杂度是衡量算法性能的数学指标，主要包括时间复杂度与空间复杂度。时间复杂度描述算法执行所需的基本操作次数，反映算法运行效率；空间复杂度描述算法在运行过程中所占用的内存空间。常见的复杂度包括 $O(1)$ 、 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 等，用于估算输入规模为 n 时算法的增长趋势。

说明：以上内容用于模型原理阐释、路径可视化处理与复杂度分析，所有回答均经过人工验证与重构，仅作为技术参考辅助，不构成最终建模决策依据，符合竞赛对 AI 工具使用的合规要求。